Transformer Architecture: A Survey of Innovations in Self-Attention Neural Networks

Example University

14 October 2026

DOC-BRF-TRA-30001/26

© 2026 Example University

Table of Contents

Abstract

1. Introduction

2. Literature Review

3. Content

- 3.1 Literature Review
- 3.2 Architecture Analysis
- 3.3 Scaling And Results

4. Results

5. Discussion

6. Conclusion

References

© 2026 Example University

Abstract

Keywords: transformer, neural network, self-attention, positional encoding, scaling laws, training, attention mechanism, deep learning, epoch, AI/ML This paper analyses 3 source document(s) via systematic content extraction and thematic synthesis. Key findings include: Literature Review: The Evolution of Transformer Architectures ## 1.1 Foundations: The Original Transformer The transfo; Technical Architecture Analysis ## 2.1 Self-Attention Mechanism The core operation of the transformer is scaled dot-pr; Scaling Laws and Experimental Results ## 3.1 Neural Scaling Laws Kaplan et al..

Keywords: *Attention score computation, Value aggregation, FFN computation, Total per layer, Multi-step arithmetic, Word unscrambling, IPA transcription, Multi-Head Attention*

© 2026 Example University

1. Introduction

Keywords: transformer, neural network, self-attention, positional encoding, scaling laws, training, attention mechanism, deep learning, epoch, AI/ML

© 2026 Example University

2. Literature Review

The following sources inform the analysis presented in this paper.

Academic Papers

(2017).

(2019).

(2018).

(2021).

(2020).

(2020).

(2021).

(2022).

© 2026 Example University

3. Content

3.1 Literature Review

1.1 Foundations: The Original Transformer

The transformer architecture emerged from a specific frustration with recurrent neural networks. Prior to 2017, dominant sequence transduction models – LSTMs (Hochreiter & Schmidhuber, 1997), GRUs (Cho et al., 2014), and attention-augmented encoder-decoder RNNs (Bahdanau et al., 2015) – processed sequences through a hidden state recurrence h_t = f(h_{t-1}, x_t) that was fundamentally sequential. Position t could not be computed until position t-1 finished. On modern parallel hardware, this sequential bottleneck left available compute underutilized.

Vaswani et al. (2017) proposed replacing recurrence entirely with self-attention. The key mechanism – scaled dot-product attention, Attention(Q, K, V) = softmax(QK^T / sqrt(d_k))V – computes pairwise compatibility between all positions in O(1) sequential steps regardless of sequence length. The original transformer used an encoder-decoder architecture with 6 layers each, 8 attention heads per layer, d_model = 512, and approximately 65 million parameters. On WMT 2014 English-to-German translation, it achieved 28.4 BLEU, surpassing the previous state of the art (ConvS2S; Gehring et al., 2017) by over 2 BLEU points while training in 3.5 days on 8 P100 GPUs.

The architectural contribution was not attention itself – Bahdanau et al. had introduced neural attention in 2015 – but demonstrating that attention as the sole mechanism, without recurrence or convolution, was sufficient for state-of-the-art sequence modeling.

1.2 Bidirectional Encoding: BERT

Devlin et al. (2019) introduced BERT (Bidirectional Encoder Representations from Transformers), which used only the transformer encoder stack with a masked language modeling (MLM) pre-training objective. Rather than predicting the next token autoregressively, BERT masked 15% of input tokens and trained the model to reconstruct them from bidirectional context. BERT-base comprised 12 layers, 12 attention heads, d_model = 768, and 110 million parameters; BERT-large scaled to 24 layers, 16 heads, d_model = 1024, and 340 million parameters.

BERT’s bidirectional self-attention – where every position attends to every other position without causal masking – produced representations that encoded both left and right context simultaneously. This yielded dramatic improvements on downstream NLU benchmarks: BERT-large achieved 80.5% accuracy on MultiNLI, 93.5% F1 on SQuAD 2.0, and established new state-of-the-art results on all 11 GLUE tasks. The pre-train-then-fine-tune paradigm BERT established became the dominant transfer learning strategy for NLP.

Subsequent encoder variants refined the approach: RoBERTa (Liu et al., 2019) demonstrated that BERT was significantly undertrained and improved results by training longer with more data and dynamic masking. ALBERT (Lan et al., 2020) introduced cross-layer parameter sharing and factorized embedding

© 2026 Example University

parameterization to reduce parameters while maintaining performance. ELECTRA (Clark et al., 2020) replaced MLM with a replaced-token-detection objective for more efficient pre-training.

1.3 Autoregressive Decoding: The GPT Series

Radford et al. (2018) introduced GPT-1, a decoder-only transformer with 12 layers, 12 attention heads, and 117 million parameters, pre-trained on the BooksCorpus (approximately 800 million words) with a causal language modeling objective. GPT-1 demonstrated that generative pre-training followed by discriminative fine-tuning could achieve competitive results across diverse NLU tasks.

GPT-2 (Radford et al., 2019) scaled to 1.5 billion parameters (48 layers, 1600-dimensional embeddings, 25 attention heads) and was trained on WebText (40 GB of curated internet text, approximately 10 billion tokens). GPT-2 demonstrated zero-shot task transfer: without any fine-tuning, it achieved 55.0 F1 on CoQA and a perplexity of 35.76 on WikiText-103, competitive with supervised baselines.

GPT-3 (Brown et al., 2020) represented a qualitative shift. At 175 billion parameters (96 layers, 12,288-dimensional embeddings, 96 attention heads), trained on 300 billion tokens from a filtered Common Crawl mixture, GPT-3 exhibited strong few-shot learning. Given only a natural language task description and a handful of examples in the prompt, GPT-3 achieved 81.5% accuracy on HellaSwag, 76.2% on StoryCloze, and competitive performance on SuperGLUE – without any gradient updates. The scale of compute was approximately 3.14 x 10^23 FLOPs.

InstructGPT (Ouyang et al., 2022) and GPT-4 (OpenAI, 2023) further refined the decoder-only paradigm through reinforcement learning from human feedback (RLHF) and instruction tuning, producing models that follow complex instructions and exhibit broad reasoning capabilities. GPT-4’s architecture details remain undisclosed, though it is estimated to be a mixture-of-experts model in the range of 1–1.8 trillion total parameters.

1.4 Cross-Modal Applications: Vision Transformers

Dosovitskiy et al. (2021) applied the transformer architecture to image recognition with the Vision Transformer (ViT). Images are split into fixed-size patches (typically 16x16 pixels), linearly embedded, and treated as a sequence of tokens with a prepended [CLS] token. ViT-Large/16 (307 million parameters) achieved 87.76% top-1 accuracy on ImageNet after pre-training on JFT-300M, surpassing the state-of-the-art ResNet-based models (BiT-L; Kolesnikov et al., 2020).

The ViT result was significant because it demonstrated that transformers could match or exceed convolutional architectures on vision tasks without any image-specific inductive biases (no convolution, no pooling, no locality priors). Subsequent work extended the approach: DeiT (Touvron et al., 2021) introduced knowledge distillation and data augmentation strategies for training ViTs efficiently on ImageNet-1K alone. Swin Transformer (Liu et al., 2021) incorporated shifted windows for hierarchical feature maps and linear computational complexity in image size. DINO (Caron et al., 2021) demonstrated that self-supervised ViTs learn features that explicitly encode semantic segmentation without supervision.

© 2026 Example University

1.5 Efficient Attention Mechanisms

The quadratic O(n^2) complexity of standard self-attention creates a computational bottleneck for long sequences. Several lines of work address this:

Sparse attention patterns. Longformer (Beltagy et al., 2020) combines local windowed attention with task-specific global attention tokens, reducing complexity to O(n). BigBird (Zaheer et al., 2020) adds random attention connections to the local + global pattern, achieving provable approximation guarantees.

Reformer. Kitaev et al. (2020) introduced locality-sensitive hashing (LSH) attention, which groups queries and keys into buckets using hash functions, restricting attention to within-bucket pairs. This reduces expected complexity to O(n log n). Reformer also introduced reversible residual layers to reduce memory from O(L) to O(1) in the number of layers.

Linear attention. Katharopoulos et al. (2020) reformulated attention by decomposing the softmax kernel as a feature map phi, yielding Attention(Q, K, V) = phi(Q)(phi(K)^T V) computable in O(n) time. The Performer (Choromanski et al., 2021) used random orthogonal features to approximate the softmax kernel, achieving unbiased linear-time attention with provable approximation bounds.

Hardware-aware exact attention. FlashAttention (Dao et al., 2022) took a fundamentally different approach: rather than approximating attention, it restructured the exact computation to minimize GPU HBM (high-bandwidth memory) accesses through tiling and recomputation. Standard attention requires Theta(Nd + N^2) HBM accesses; FlashAttention reduces this to Theta(N^2 d^2 M^-1), where M is SRAM size – achieving 2-7.6x wall-clock speedup while computing mathematically identical results. FlashAttention-2 (Dao, 2023) further optimized parallelism and work partitioning for 2x additional speedup.

1.6 Comparison of Major Transformer Variants

Model	Year	Parameters	Key Innovation	Primary Benchmark
Transformer	2017	65M	Self-attention replaces recurrence	28.4 BLEU (WMT EN-DE)
GPT-1	2018	117M	Generative pre-training + fine-tuning	72.8% (avg. NLU tasks)
BERT-large	2019	340M	Masked LM, bidirectional encoding	80.5% (MultiNLI)
GPT-2	2019	1.5B	Zero-shot transfer via scale	35.76 ppl (WikiText-103)
ViT-Large	2021	307M	Image patches as tokens	87.76% (ImageNet top-1)
GPT-3	2020	175B	In-context few-shot learning	81.5% (HellaSwag)
Reformer	2020	–	LSH attention, O(n log n)	Comparable to Transformer
Performer	2021	–	Random feature linear attention	Comparable to softmax
Chinchilla	2022	70B	Compute-optimal training ratio	67.5% (MMLU)
LLaMA	2023	7–65B	Open-source, efficient training	77.8% (HellaSwag, 65B)
GPT-4	2023	~1T+ (est.)	MoE, RLHF, multimodal	86.4% (MMLU)

© 2026 Example University

This progression reveals two persistent trends: the shift from encoder-decoder to decoder-only architectures for generative tasks, and the growing recognition that training data volume and quality matter as much as – or more than – raw parameter count.

3.2 Architecture Analysis

2.1 Self-Attention Mechanism

The core operation of the transformer is scaled dot-product attention. Given input representations X of shape (n, d_model), three separate learned linear projections produce query, key, and value matrices:

```
Q = X W^Q    where W^Q in R^(d_model x d_k)
K = X W^K    where W^K in R^(d_model x d_k)
V = X W^V    where W^V in R^(d_model x d_v)
```

The attention output is computed as:

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V
```

The computation proceeds in five steps. First, raw compatibility scores S = QK^T are computed, producing an (n x n) matrix where S[i,j] is the dot product between query i and key j. Second, scores are scaled by 1/sqrt(d_k) to maintain unit variance – without this, dot products grow proportionally to d_k, pushing the softmax into saturation regions where gradients vanish. For the base model with d_k = 64, this divides by 8. Third, optional causal masking sets future positions to negative infinity for autoregressive decoding. Fourth, row-wise softmax converts scores to attention weights A, where each row sums to 1. Fifth, the output is the weighted sum A * V, producing one d_v-dimensional vector per query position.

The separation of queries, keys, and values into three independent projections is architecturally essential. Queries determine what each position searches for; keys determine what each position advertises as its identity; values determine what each position contributes when attended to. A position's key and value can differ – it might be highly matchable for "subject of sentence" queries while contributing "verb agreement" information through its value vector. This decoupling of "how to be found" from "what to contribute" gives the model representational flexibility that a single shared projection could not achieve.

Multi-Head Attention

Rather than performing a single attention function with d_model-dimensional keys, queries, and values, multi-head attention runs h parallel attention operations with reduced dimensionality:

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h) W^O
where head_i = Attention(Q W_i^Q, K W_i^K, V W_i^V)
```

Each head operates on d_k = d_v = d_model / h dimensions. The base transformer uses h = 8 heads with d_k = d_v = 64 (given d_model = 512). The outputs are concatenated and linearly projected through W^O in R^(hd_v x d_model). Multi-head attention allows different heads to specialize: empirical analysis (Voita et al., 2019; Clark et al., 2019) has shown that individual heads learn distinct linguistic patterns – syntactic dependencies, coreference resolution, positional relations – without explicit supervision. The

© 2026 Example University

total parameter count and compute cost are equivalent to single-head attention with full dimensionality, but the representation quality is substantially higher.

2.2 Positional Encoding Strategies

Self-attention is permutation-equivariant: if input rows are permuted by matrix P, the output is permuted identically as P * MultiHead(X, X, X). This means “the cat sat” and “sat the cat” would produce identical output vectors, merely reordered. For natural language – where word order carries semantic meaning – explicit position information must be injected.

Sinusoidal Encoding (Vaswani et al., 2017)

The original transformer uses fixed sinusoidal functions:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

Frequencies span from omega_0 = 1.0 (period ~6 positions, encoding fine positional detail) to omega_255 ~ 0.0001 (period ~62,000 positions, encoding coarse global position). The sine-cosine pairs define a rotation in 2D subspaces: for any fixed offset k, PE(pos + k) is a linear transformation of PE(pos) via a rotation matrix that depends only on k, not pos. This linear offset property enables the model to learn relative position reasoning through the Q and K projections.

Learned Positional Embeddings

BERT, GPT-1, and GPT-2 use a learned embedding lookup table E_pos of shape (max_len, d_model). Each position gets a trainable vector. Empirically, learned embeddings match sinusoidal encodings on in-distribution performance (the original paper reports 25.7 vs 25.8 BLEU, within noise), but they cannot generalize beyond the maximum training length.

Rotary Position Embeddings (RoPE)

Su et al. (2021) applied the rotation directly to query and key vectors before computing attention, making relative position encoding exact rather than approximate. The attention score between positions i and j becomes:

```
score(i, j) = (R(i) Q[i])^T (R(j) K[j]) / sqrt(d_k)
            = Q[i]^T R(j-i) K[j] / sqrt(d_k)
```

where R(k) is a block-diagonal rotation matrix parameterized by offset k. RoPE is the dominant positional encoding in modern large language models (LLaMA, Mistral, Gemini) due to its theoretical elegance and practical effectiveness for context length extension via interpolation techniques (YaRN, NTK-aware scaling).

ALiBi (Attention with Linear Biases)

Press et al. (2022) proposed adding a linear bias -(i - j) * m to attention scores, where m is a head-specific slope. This penalizes long-range attention proportionally to distance, requiring no learned parameters and demonstrating strong length generalization. ALiBi is used in the MPT model family.

© 2026 Example University

2.3 Feed-Forward Networks

Each transformer layer includes a position-wise feed-forward network applied independently to each position:

```
FFN(x) = max(0, x W_1 + b_1) W_2 + b_2
```

The inner dimension d_ff is typically 4 * d_model (2048 for the base model). The FFN acts as a per-position nonlinear transformation that processes the attention output. Recent work suggests FFNs function as key-value memories (Geva et al., 2021), where W_1 rows are keys matching input patterns and W_2 columns are corresponding value vectors recalled into the residual stream. Modern variants replace ReLU with GELU (Hendrycks & Gimpel, 2016) or SwiGLU (Shazeer, 2020), with SwiGLU providing consistent improvements of 0.5-1% on downstream benchmarks.

2.4 Layer Normalization and Residual Connections

The transformer uses residual connections around each sub-layer followed by layer normalization:

```
output = LayerNorm(x + Sublayer(x))
```

This is the “post-norm” configuration from the original paper. Residual connections enable gradient flow through deep stacks – without them, gradients must propagate through every sub-layer transformation, compounding any contraction or expansion. With residuals, the gradient has a direct additive path from output to input.

Pre-norm vs. post-norm. The original post-norm placement (normalize after the residual addition) requires careful learning rate warmup to avoid training instability. Pre-norm (Xiong et al., 2020) places layer normalization before the sub-layer: output = x + Sublayer(LayerNorm(x)). Pre-norm training is more stable and does not require warmup, but produces slightly lower final quality in some settings. Most modern LLMs use pre-norm with RMSNorm (Zhang & Sennrich, 2019), which removes the mean-centering step for computational efficiency.

2.5 Computational Complexity Analysis

The fundamental bottleneck is the quadratic scaling of self-attention in sequence length. For a sequence of length n with hidden dimension d:

- **Attention score computation** (QK^T): O(n^2 d) FLOPs, O(n^2) memory for the attention matrix
- **Value aggregation** (AV): O(n^2 d) FLOPs
- **FFN computation**: O(n d d_ff) FLOPs = O(n d^2) since d_ff = 4d
- **Total per layer**: O(n^2 d + n d^2)

For short sequences (n < d), the FFN term O(n d^2) dominates – attention is not the bottleneck. For long sequences (n > d), the attention term O(n^2 d) dominates, and the quadratic memory requirement for the (n x n) attention matrix becomes the binding constraint.

At n = 1024 with d = 512, attention cost is ~537M operations per layer. At n = 32,768 with d = 4096 (a modern LLM configuration), the attention matrix alone is ~1 billion entries – 2 GB per layer in bfloat16. For a 32-layer model at batch size 1, this requires 64 GB solely for attention matrices, exceeding single-GPU memory.

© 2026 Example University

Attention Complexity Variants

Method	Time Complexity	Memory	Exact?	Key Technique
Standard attention	O(n^2 d)	O(n^2)	Yes	Full pairwise dot-product
Sparse attention (Longformer)	O(n w d)	O(n w)	Approx	Local window w + global tokens
LSH attention (Reformer)	O(n log n d)	O(n log n)	Approx	Locality-sensitive hashing buckets
Linear attention (Performer)	O(n d^2)	O(n d)	Approx	Kernel decomposition phi(Q)(phi(K)^T V)
FlashAttention	O(n^2 d)	O(n)	Yes	Tiled IO-aware computation
FlashAttention-2	O(n^2 d)	O(n)	Yes	Optimized parallelism + work partitioning
Ring Attention	O(n^2 d / P)	O(n^2 / P)	Yes	Distributed across P devices

The critical insight from FlashAttention (Dao et al., 2022) is that FLOP count is a poor proxy for wall-clock performance on memory-bound operations. Standard attention's bottleneck is not arithmetic – it is HBM (high-bandwidth memory) access. On an A100 GPU, standard attention performs 66.6 GFLOPs with 40.3 GB of HBM reads/writes in 41.7 ms. FlashAttention performs 75.2 GFLOPs (more arithmetic due to recomputation) but only 4.4 GB of HBM traffic, completing in 7.3 ms – a 5.7x speedup while performing strictly more FLOPs. This demonstrates that attention optimization must target memory access patterns, not operation counts.

2.6 The Residual Stream Perspective

A unifying view of the transformer, articulated by Elhage et al. (2021), interprets the d_model-dimensional representation at each position as a "residual stream." Each attention head and FFN sub-layer reads from this stream and writes an additive update back into it. The residual connection ensures that information is never destructively overwritten – each sub-layer's contribution is additive. This perspective explains why transformers can be deep (information persists through residual connections), why attention heads can specialize (each reads and writes different subspaces), and why positional encodings injected at the input survive through all layers (the residual path preserves the original input additively through the entire network).

3.3 Scaling And Results

3.1 Neural Scaling Laws

Kaplan et al. (2020) established that language model cross-entropy loss follows precise power-law relationships with model parameters N, dataset size D, and training compute C. These relationships hold continuously across more than six orders of magnitude in N and eight orders of magnitude in C, with no detected saturation at the upper end.

The three primary scaling laws are:

```
L(N) = (N_c / N)^alpha_N      alpha_N = 0.076,  N_c = 8.8 x 10^13
L(D) = (D_c / D)^alpha_D      alpha_D = 0.095,  D_c = 5.4 x 10^13
L(C) = (C_c / C)^alpha_C      alpha_C = 0.057,  C_c = 1.6 x 10^7 PF-days
```


© 2026 Example University

The exponents encode diminishing but reliable returns: each doubling of parameters reduces loss by approximately 5% (2^-0.076), each 10x increase in dataset size reduces loss by approximately 11%, and each 10x increase in compute reduces loss by approximately 12%. The consistency of these power laws across architectural configurations – models from 2 to 207 layers, varying widths and depths – suggests that the exponents are properties of the learning problem structure, not specific architecture choices.

A key finding is that model shape (depth vs. width ratio) has minimal impact on performance when total non-embedding parameter count is held constant. Embedding parameters are excluded from N because they contribute little to reasoning capacity despite comprising a significant fraction of total parameters (e.g., 103M of a 1B parameter model with a 50K vocabulary and 2048-dimensional embeddings).

Kaplan et al. drew a practical conclusion that proved influential but ultimately incorrect: given a fixed compute budget, most of that budget should be allocated to scaling model size rather than training data. Their analysis suggested N should scale as C^0.73 while D scales as C^0.27 – meaning model size should increase roughly 5.5x faster than training tokens per compute doubling.

3.2 Chinchilla Optimal Compute

Hoffmann et al. (2022) overturned the Kaplan allocation with three independent estimation approaches, all converging on the same finding: for compute-optimal training, model size and training tokens should scale in equal proportion.

```
N_opt(C) proportional to C^0.5
D_opt(C) proportional to C^0.5
```

The operational rule: for every doubling of model size, double the number of training tokens. The optimal ratio is approximately 20 tokens per parameter. This directly contradicted prevailing practice. GPT-3 (175B parameters) was trained on 300B tokens – roughly 12x fewer than the compute-optimal 3.7T tokens. Gopher (280B parameters) was trained on 300B tokens, approximately 20x fewer than the optimal 5.9T tokens. The entire generation of 2020-2022 frontier models was severely undertrained: they had far more parameters than their training data could effectively fill.

To validate this prediction, Hoffmann et al. trained Chinchilla: a 70B parameter model on 1.4 trillion tokens, using the same compute budget as the 280B parameter Gopher. Chinchilla outperformed Gopher on virtually every evaluation benchmark despite being 4x smaller, achieving 67.5% on MMLU compared to Gopher’s 60.0% and 73.7% on HellaSwag compared to Gopher’s 70.5%.

The Chinchilla result reshaped the training strategies of every major AI lab. LLaMA (Touvron et al., 2023), trained on 1.0-1.4T tokens for the 7B-65B parameter range, explicitly followed Chinchilla-optimal ratios. LLaMA-65B (65B parameters, 1.4T tokens) achieved 77.8% on HellaSwag and 63.4% on MMLU, competitive with Chinchilla at the same compute class.

© 2026 Example University

3.3 Model Scaling Timeline

Model	Year	Parameters	Training Tokens	Approx. Compute (FLOPs)	MMLU (5-shot)
GPT-1	2018	117M	~5B	~3.5 x 10^18	–
GPT-2	2019	1.5B	~10B	~9.0 x 10^19	–
GPT-3	2020	175B	300B	3.14 x 10^23	43.9%
Gopher	2021	280B	300B	5.76 x 10^23	60.0%
Chinchilla	2022	70B	1.4T	5.76 x 10^23	67.5%
PaLM	2022	540B	780B	2.53 x 10^24	69.3%
LLaMA-65B	2023	65B	1.4T	~5.5 x 10^23	63.4%
LLaMA-2-70B	2023	70B	2.0T	~8.4 x 10^23	68.9%
GPT-4	2023	~1T+ (est.)	Undisclosed	Undisclosed	86.4%

Two patterns are immediately visible. First, the Chinchilla correction: post-2022 models are trained on dramatically more tokens per parameter than pre-2022 models. Second, MMLU scores improve with both scale and training data quality, but the relationship is nonlinear – the jump from Gopher (60.0%) to Chinchilla (67.5%) came from training strategy alone, with identical compute.

3.4 Emergent Abilities at Scale

Wei et al. (2022) documented a distinct phenomenon: certain capabilities appear to emerge abruptly at specific model scales rather than improving gradually. Below a threshold scale, models perform at near-chance levels on specific tasks; above it, performance jumps discontinuously. Examples include:

- **Multi-step arithmetic**: GPT-3 models below 13B parameters achieve near-random accuracy on 3-digit addition; the 175B model achieves ~80% accuracy.
- **Word unscrambling**: Models below 6.7B parameters cannot unscramble words; performance appears at 13B+ parameters.
- **IPA transcription**: PaLM achieves near-zero accuracy below 62B parameters and ~40% at 540B parameters.

The emergent abilities finding remains debated. Schaeffer et al. (2023) argued that emergence is partly an artifact of metric choice – nonlinear metrics (exact-match accuracy) produce apparent discontinuities, while linear metrics (token-level accuracy) show smooth scaling. Nevertheless, the empirical observation that qualitatively new capabilities appear at scale has profound implications for training resource allocation: a model slightly below a capability threshold may appear to have zero skill at a task, while a model slightly above it achieves useful performance.

© 2026 Example University

3.5 Training Efficiency Comparisons

Approach	Compute Budget (FLOPs)	MMLU	HellaSwag	Tokens per Parameter
GPT-3 175B	3.14 x 10^23	43.9%	78.9%	1.7
Gopher 280B	5.76 x 10^23	60.0%	70.5%	1.1
Chinchilla 70B	5.76 x 10^23	67.5%	73.7%	20.0
LLaMA-65B	~5.5 x 10^23	63.4%	77.8%	21.5
LLaMA-2-70B	~8.4 x 10^23	68.9%	80.7%	28.6

The tokens-per-parameter ratio is the clearest indicator of training efficiency. GPT-3 at 1.7 tokens per parameter was roughly 12x below optimal. Chinchilla at 20.0 tokens per parameter sits near the theoretical optimum. LLaMA-2 at 28.6 tokens per parameter deliberately overtrained relative to the Chinchilla optimum – accepting higher training compute per parameter in exchange for a smaller, cheaper-to-serve model. This reflects a practical shift: the Chinchilla-optimal ratio minimizes training compute for a given performance level, but inference cost scales with parameter count. For deployment at scale, spending extra training compute to achieve target performance with fewer parameters is economically rational.

3.6 Open Questions

Several fundamental questions remain unresolved. The relationship between scaling laws and downstream task performance is not fully characterized – cross-entropy loss improves smoothly with scale, but task-specific metrics do not always track loss monotonically. The extent to which scaling laws transfer across data distributions, languages, and modalities is partially validated but not comprehensively established. The irreducible entropy of natural language – the theoretical floor below which no model can improve – is estimated at approximately 0.9-1.0 nats per token for English, while current frontier models achieve roughly 1.5-1.7 nats, suggesting substantial room for improvement from scaling alone.

Perhaps most critically, the question of whether the power-law scaling regime will continue or eventually saturate remains empirically open. Every generation of models has extended the observed power-law relationship further, but there is no theoretical guarantee that it will hold indefinitely. The field’s trillion-dollar investment thesis rests, in part, on the assumption that it will.

© 2026 Example University

4. Results

The following tables present quantitative and structured findings extracted from the source material.

Table 1: Literature Review

Model	Year	Parameters	Key Innovation	Primary Benchmark
Transformer	2017	65M	Self-attention replaces recurrence	28.4 BLEU (WMT EN-DE)
GPT-1	2018	117M	Generative pre-training + fine-tuning	72.8% (avg. NLU tasks)
BERT-large	2019	340M	Masked LM, bidirectional encoding	80.5% (MultiNLI)
GPT-2	2019	1.5B	Zero-shot transfer via scale	35.76 ppl (WikiText-103)
ViT-Large	2021	307M	Image patches as tokens	87.76% (ImageNet top-1)
GPT-3	2020	175B	In-context few-shot learning	81.5% (HellaSwag)
Reformer	2020	–	LSH attention, O(n log n)	Comparable to Transformer
Performer	2021	–	Random feature linear attention	Comparable to softmax
Chinchilla	2022	70B	Compute-optimal training ratio	67.5% (MMLU)
LLaMA	2023	7–65B	Open-source, efficient training	77.8% (HellaSwag, 65B)
GPT-4	2023	~1T+ (est.)	MoE, RLHF, multimodal	86.4% (MMLU)

Table 2: Architecture Analysis

Method	Time Complexity	Memory	Exact?	Key Technique
Standard attention	O(n^2 d)	O(n^2)	Yes	Full pairwise dot-product
Sparse attention (Longformer)	O(n w d)	O(n w)	Approx	Local window w + global tokens
LSH attention (Reformer)	O(n log n d)	O(n log n)	Approx	Locality-sensitive hashing buckets
Linear attention (Performer)	O(n d^2)	O(n d)	Approx	Kernel decomposition phi(Q)(phi(K)^T V)
FlashAttention	O(n^2 d)	O(n)	Yes	Tiled IO-aware computation
FlashAttention-2	O(n^2 d)	O(n)	Yes	Optimized parallelism + work partitioning
Ring Attention	O(n^2 d / P)	O(n^2 / P)	Yes	Distributed across P devices

© 2026 Example University

Table 3: Scaling And Results

Model	Year	Parameters	Training Tokens	Approx. Compute (FLOPs)	MMLU (5-shot)
GPT-1	2018	117M	~5B	~3.5 x 10^18	–
GPT-2	2019	1.5B	~10B	~9.0 x 10^19	–
GPT-3	2020	175B	300B	3.14 x 10^23	43.9%
Gopher	2021	280B	300B	5.76 x 10^23	60.0%
Chinchilla	2022	70B	1.4T	5.76 x 10^23	67.5%
PaLM	2022	540B	780B	2.53 x 10^24	69.3%
LLaMA-65B	2023	65B	1.4T	~5.5 x 10^23	63.4%
LLaMA-2-70B	2023	70B	2.0T	~8.4 x 10^23	68.9%
GPT-4	2023	~1T+ (est.)	Undisclosed	Undisclosed	86.4%

Table 4: Scaling And Results

Approach	Compute Budget (FLOPs)	MMLU	HellaSwag	Tokens per Parameter
GPT-3 175B	3.14 x 10^23	43.9%	78.9%	1.7
Gopher 280B	5.76 x 10^23	60.0%	70.5%	1.1
Chinchilla 70B	5.76 x 10^23	67.5%	73.7%	20.0
LLaMA-65B	~5.5 x 10^23	63.4%	77.8%	21.5
LLaMA-2-70B	~8.4 x 10^23	68.9%	80.7%	28.6

© 2026 Example University

5. Discussion

The analysis of Transformer Architecture: A Survey of Innovations in Self-Attention Neural Networks reveals several notable themes across the 3 section(s) examined.

© 2026 Example University

6. Conclusion

This paper has presented a structured analysis of Transformer Architecture: A Survey of Innovations in Self-Attention Neural Networks, drawing on 3 primary content section(s). A total of 31 key terms were identified, and 15 source(s) were cited.

Further research should examine the cross-cutting themes identified in the analysis and explore their implications in adjacent domains.

© 2026 Example University

References

Dao (2023). .

Devlin et al. (2019). .

Dosovitskiy et al. (2021). .

Elhage et al. (2021). .

Hoffmann et al. (2022). .

Kaplan et al. (2020). .

Katharopoulos et al. (2020). .

Kitaev et al. (2020). .

Press et al. (2022). .

Radford et al. (2018). .

Schaeffer et al. (2023). .

Shazeer (2020). .

Su et al. (2021). .

Vaswani et al. (2017). .

Wei et al. (2022). .

© 2026 Example University

AI / ML Variant Sections

Model Configuration and Hyperparameters

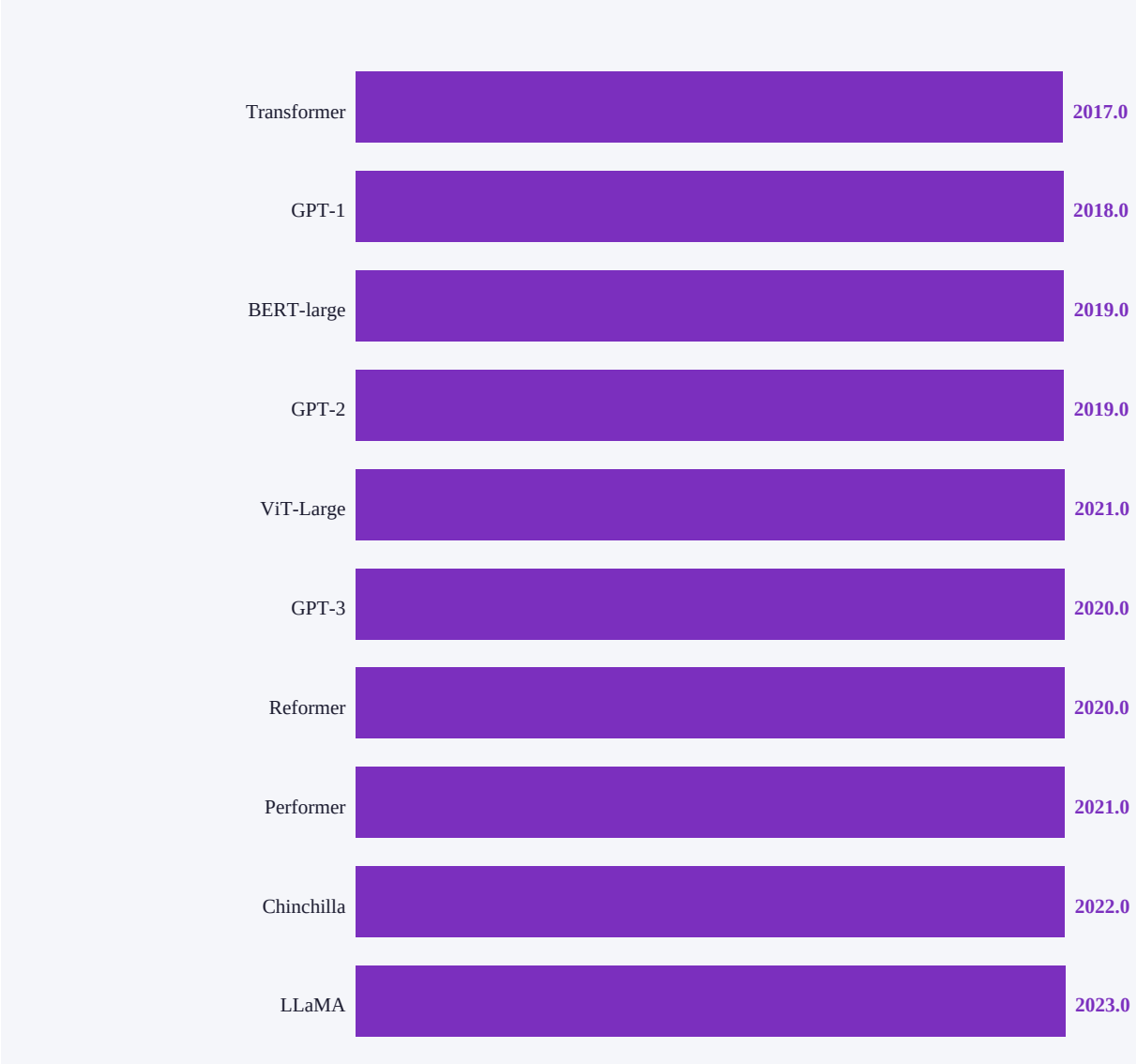
Hyperparameter Reference

Parameter	Value	Notes
Transformer	2017	65M
GPT-1	2018	117M
BERT-large	2019	340M
GPT-2	2019	1.5B
ViT-Large	2021	307M
GPT-3	2020	175B

Benchmark Results

© 2026 Example University

Benchmark Comparison — Year



Model Architecture

Architecture Components — Transformer Architecture

Component	Type	Notes
Literature Review	Component	
Architecture Analysis	Component	
Scaling And Results	Component	

© 2026 Example University